

v/presentation/d/1LXcdqyd6U_FlUprPVvol3GldmYebfwyL/edit?slide=id.p1#slide=id.p1

View only

Slideshow Share

 **CODING HUB**
Innovations Pvt. Ltd.
Way to a better future

COLLECTION FRAMEWORK



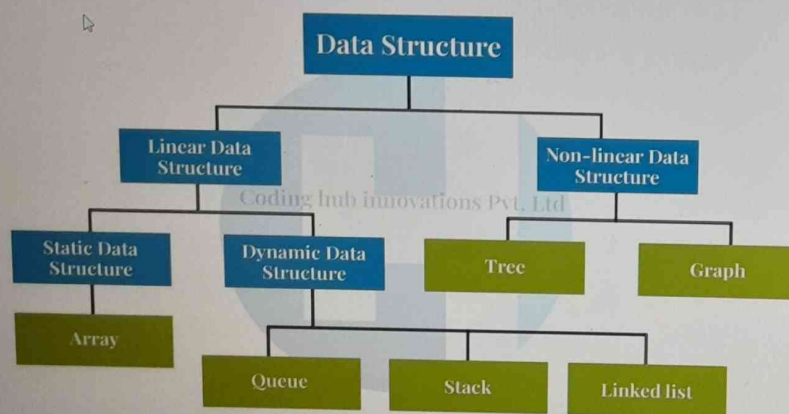
CODING HUB INNOVATIONS PVT LTD

What is a Data Structure ?

1. **A data structure is a storage that is used to store and organize data. It is a way of arranging data on a computer so that it can be accessed and updated efficiently.**
2. **There are two types of Data Structures**
 - a. **Linear Data Structures stores the data in sequential order**
 - b. **Non Linear Data Structures stores the data in non-sequential order**



Classification of Data Structure



Array

1. Definition : An array is an Object in java which is used to store homogeneous data
2. An array is a Linear data structure which stores the data in sequential order
3. Arrays uses indices to store the element sequentially

Advantages of Arrays

1. Array can hold multiple values in a single variable
2. Random access
3. Array operations are so fast
 - a. Performance will be so high
4. Searching an element in the array is so easy
5. Array will hold both primitive and non-primitive data types

DISADVANTAGES OF ARRAYS

1. Arrays can hold homogeneous data only
2. Using Arrays memory will be wasted
3. Arrays are static in nature
 - a. once we declare the size of the array we can't increase or decrease the size of the array
4. Insertion and deletion is difficult.

UNDERSTANDING OBJECT ARRAY

```
1 package com.codinghub.p2;
2 import java.io.IOException;
7 public class Three
8 {
9     public static void main(String args[])
10    {
11        //PRIMITIVE ARRAY
12        int[] arr=new int[] {1,2,3};
13        //NON PRIMITIVE PRE-DEFINED CLASS ARRAY
14        String a[]=new String[] {"hello","all"};
15        Student s1=new Student();
16        Student s2=new Student();
17        Student s3=new Student();
18        Student s4=new Student();
19        //NON PRIMITIVE USER DEFINED CLASS ARRAY
20        Student s[]=new Student[] {s1,s2,s3,s4};
21        //HOLDS BOTH PRIMITIVE AND NON PRIMITIVE
22        Object obj[]= {1,1.5,1.5f,true,"hello",'x',s1,s2,s3};
23    }
24 }
25 class Student
26 {
27 }
```

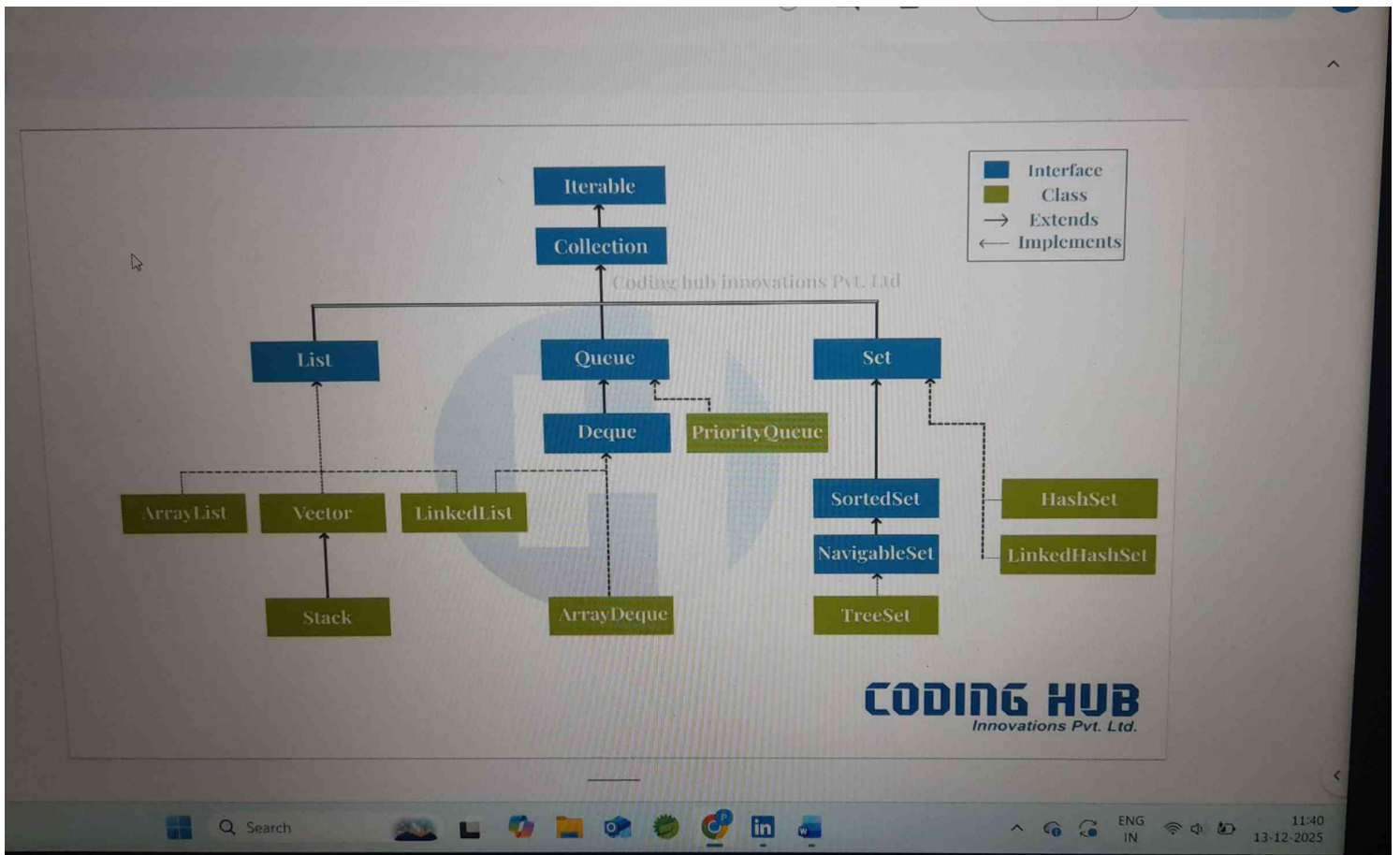
COLLECTION & COLLECTION FRAMEWORK

1. Collection(Interface **Java.util**) : A group of objects represented as a single unit is called Collection
 1. Collection interface is the root interface of Collection Framework
 2. Collections Framework consists of several classes & interfaces, which can be used to represent a group of objects as a single unit

COLLECTION

Definition: A group of individual objects represented as a single unit is called collection.

1. Collections can hold both homogeneous and heterogeneous data
2. Collections are dynamic in nature
3. Collection will hold only non-primitive data types
4. Performance point of view arrays are best to use when compared to collections



List Interface

List : It is a interface that is present in java.util package

1. It is the child interface of Collection interface
2. Implementation Classes
--> ArrayList , LinkedList , Vector & Stack
3. List will accept duplicate values
4. List will preserve the insertion order
5. Null insertion is possible
6. Random access is there using get()
7. Legacy classes are Stack & Vector
8. We can create a List Object using
`List list=new ArrayList();`
9. List will take both Homogeneous and Heterogeneous data

Set Interface

Set : It is a interface that is present in the java.util package

1. It is the child interface of Collection interface
2. Implementation classes are
 - > HashSet ,LinkedHashSet & TreeSet
3. Set interface may or may not take Heterogeneous data
 - > TreeSet = Accepts only Homogeneous data
 - > HashSet & LinkedSet = Accepts both Homogeneous & Heterogeneous data
4. Null insertion is may or may not be possible
 - > TreeSet = Do not accept null values
 - > HashSet & LinkedSet = Accepts null values
5. Do not allow duplicate values
6. Insertion order may or may not be preserved
 - > LinkedHashSet = Preserve the Insertion order
 - > HashSet & TreeSet = Do not preserve the insertion order
7. We can not access elements Randomly

Queue Interface

Queue : It is a interface that is present in the java.util package

1. It is the child interface of Collection Framework
2. Implementation classes are
 - > PriorityQueue & ArrayDeque
3. Queue may or may not accept Heterogeneous data
 - > Priority Queue = Homogeneous data only
 - > ArrayDeque = Homogeneous + Heterogeneous data
4. Duplicate elements will be added
5. Insertion order may or may not be preserved
 - PriorityQueue = Not preserved
 - ArrayDeque = preserve
6. Null Insertion is not possible
7. Random access is not possible

Underlying Data Structures

Underlying DataStructures :

1. ArrayList = Dynamic Array / Resizable Array
2. LinkedList = Doubly Linked List
3. Vector = Dynamic Array / Resizable Array
4. Stack = Implement by using Array or LinkedList
5. Priority Queue = Binary Heap
6. ArrayDeque = Dynamic Array / Resizable Array
7. HashSet = Hash Table
8. LinkedHashSet : HashSet + doubly LinkedList
9. TreeSet : TreeMap

Wrapper Classes

Wrapper

converts a integer value to object

int

Primitive Data Type

int x=10;
int y=20;

Integer x=10;
Integer y=20;

Wrapper Class

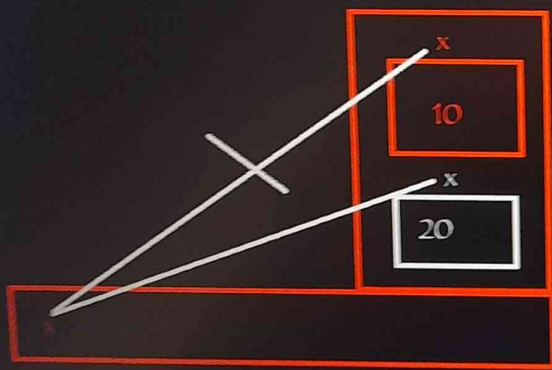
int

class Integer
Non-Primitive Data Type

Wrapper Classes are immutable

```
public class Main
{
    public static void main(String[] args)
    {
        Integer x=10;
        System.out.println(x+"==>" + System.identityHashCode(x));
        x=x+10;
        System.out.println(x+"==>" + System.identityHashCode(x));
    }
}
```

Wrapper Classes are Immutable



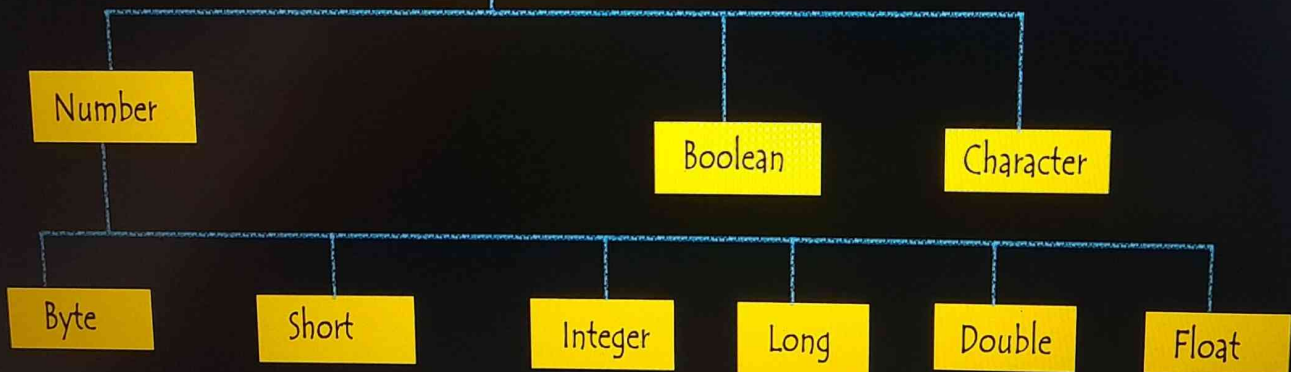
Immutability = Not changeable
if we try to change it , new object will be created

SVSC = Venkatesh

```
ArrayList<Integer> x=new Integer<>();  
LinkedList<String> s=new LinkedList<>();
```

Wrapper Class Hierarchy

Object => Super class of all the classes



BOXING

Boxing/Autoboxing : Converting Primitive Data type to a wrapper class is called Boxing

Example :

```
int x=10; //Primitive Data Type
```

```
Integer num1=x; // Primitive Data Type converted to Class
```

UnBoxing : Converting Wrapper class Object into Primitive data types is called UnBoxing

Example :

```
Integer num1=10; //Wrapper class
```

```
int x=num1; //Wrapper Class converted to Primitive Data type
```


Collection Interface Methods

1. Collection interface is the root interface of Collection hierarchy and provides basic methods
2. Operations
 - a. `add()`,
 - b. `add(index,value)`
 - c. `remove()`,
 - d. `contains()`,
 - e. `size()`,
 - f. `isEmpty()`,
 - g. `addAll()`,
 - h. `removeAll()`,
 - i. `retainAll()`
 - j. `clear()`
 - k. `containsAll()`
 - l. `iterator()`
 - m. `toArray()`
 - n. `forEach()` etc..



```
50 public static void main(String args[])
51 {
52     ArrayList<Integer> al=new ArrayList<>();
53     al.add(10); //10
54     al.add(20); //
55     al.add(30);
56     al.add(65);
57     System.out.println("IM COLLECTION OBJECT");
58     System.out.println(al);
59     System.out.println("IM FOR LOOP");
60     for(int x=0;x<al.size();x++)
61         System.out.print(al.get(x)+ " ");
62     System.out.println();
63     System.out.println("IM FOR-EACH TYPED COLLECTION LOOP");
64     for(Integer temp : al)
65         System.out.print(temp+" ");
66     System.out.println();
67     System.out.println("IM FOR-EACH LOOP IM NOT A TYPED COLLECTION");
68     for(Object temp1 : al)
69         System.out.print(temp1+" ");
70     System.out.println();
71     System.out.print("IM ITERATOR");
72     System.out.println();
73     Iterator it=al.iterator(); //
74     while(it.hasNext()) //while(true) //
75     {
76         System.out.print(it.next()+" ");
77     }
78 }
```

ARRAYLIST

1. Exactly same as Array except ArrayList is Dynamic
2. It is the implementation class for list
3. It is a Linear Data Structure & Stores the elements contiguously
4. Size is Dynamic in nature 
5. The Default capacity of arrayList is 10
6. Heterogeneous & duplicate elements can be stored
7. Null insertion is possible
8. Insertion order is preserved

ARRAYLIST

1. Once reaches the default capacity changes the size to half the current capacity
 - a. $\text{New Capacity} = \text{CurrentCapacity} + (\text{CurrentCapacity} / 2);$
2. ArrayList is used mostly used for retrieving Operation, because array list implements random access interface
3. If our operation is insertion or deletion then array list must not be used because internally array has to shift its objects
4. Because of the above disadvantage we are moving to the linked list

```
5 class Demo
6 {
7     public static void main(String args[])
8     {
9         int [] a=new int[] {10,20,10,19,10,60,70};
10        ArrayList<Integer> al=new ArrayList<>();
11        int x;
12        for(x=0;x<a.length;x++)
13        {
14            al.add(a[x]);
15        }
16        Iterator it = al.iterator();
17        while(it.hasNext())
18        {
19            System.out.println(it.next());
20        }
21    }
22 }
```

```
ArrayList<String> al=new ArrayList<>();//CREATING AL
al.add("APPLE");
al.add("MANGO");
al.add("BANANA");
al.add("COCONUT");//ADDING OBEJECTS FOR AL
System.out.println(al);
al.add(1,"PINEAPPLE");//INSERTING OBJECT IN THE AL
System.out.println(al);
al.set(2, "GRAPES");//UPDATING OBJECT IN THE AL
System.out.println(al);
al.remove(3);//DELETING OBJECT IN THE AL
System.out.println(al);
System.out.println(al.contains("APPLE"));//SEARCH
System.out.println(al.get(2));//FETCH THE OBJECT IN INDEX
System.out.println(al.indexOf("COCONUT"));//FIRST OCCURANCE
System.out.println(al.lastIndexOf("APPLE"));//LAST OCCURANCE
```

LINKED LIST(Doubly Linked List)

1. It is the implementation class for List
2. It is a linear data structure
3. Elements are not stored contiguously in the memory so that memory will not be wasted
4. Doubly linked list is used in the memory to store the elements randomly
5. Size is Dynamic in nature
6. Heterogeneous & duplicate elements can be stored
7. Null insertion is possible
8. Insertion order is preserved

Array :

1. Memory will be wasted

2. Insertion and Deletion is complex

=> 10 20 30 40 50 60 70 80



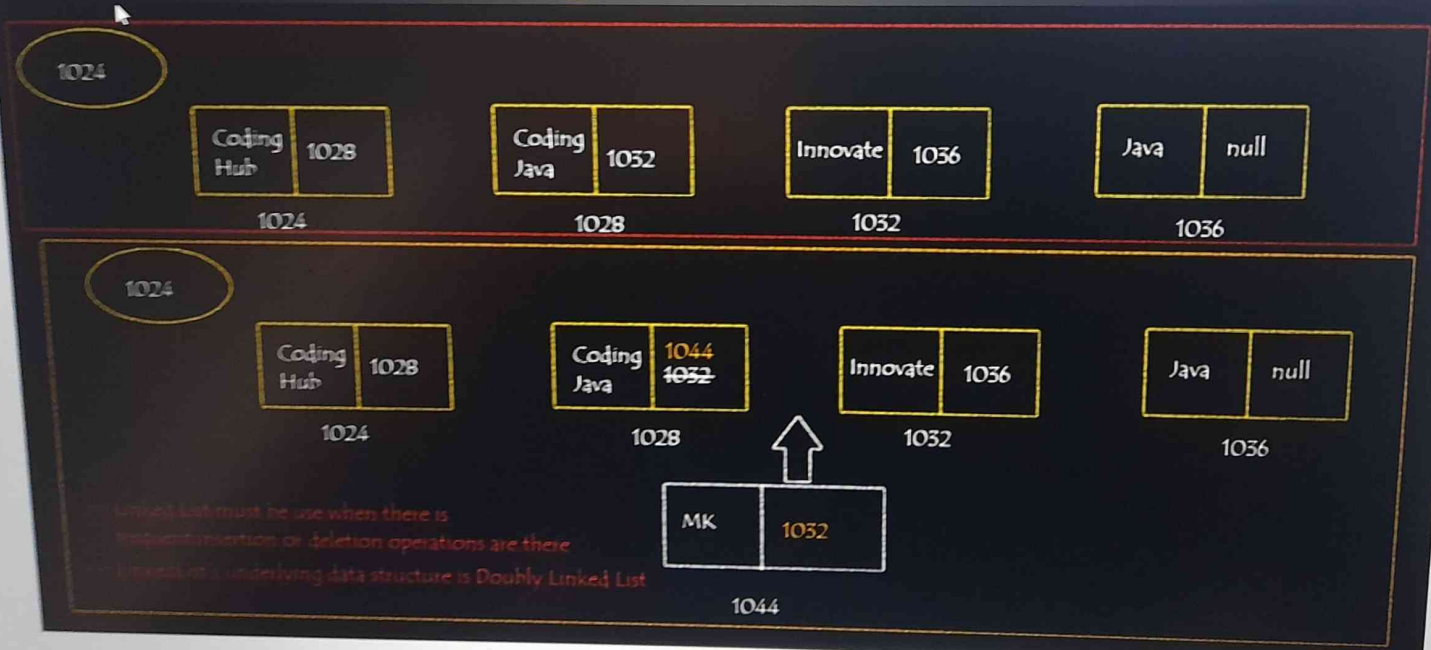
45

=> 10 20 30 40 45 50 60 70 80



Linked List

Single Linked List



Doubly Linked List

Prev Address	Data	Next Address
--------------	------	--------------

1. Instead of using Single Linked List if we use Doubly Linked List then the performance will be increased
==> Searching will be done from both the ends

null	Hello	1028
------	-------	------

1024

1024	all	1032
------	-----	------

1028

1028	good	1036
------	------	------

1032

1032	morning	null
------	---------	------

1036

LINKED LIST

- Does not implement Random Access interface
- If our operation is insertion or deletion then using linked list is the best way
- If our operation is retrieving then we must not use linked list
- Methods :
 - addFirst(), addLast, removeFirst, removeLast, getFirst, getLast

```
4 class Demo
5 {
6     public static void main(String args[])
7     {
8         int[] a=new int[] {5,8,24,2,11,32,27,21,53};
9         LinkedList l=new LinkedList();
10        for(int x=0;x<a.length;x++)
11            l.add(a[x]);
12        ListIterator ite=l.listIterator();
13        System.out.println("IN FORWARD DIRECTION");
14        while(ite.hasNext())
15        {
16            System.out.println(ite.next());
17        }
18        System.out.println("IN BACKWARD DIRECTION");
19        while(ite.hasPrevious())
20        {
21            System.out.println(ite.previous());
22        }
23    }
24 }
```

VECTOR

(UNDERLYING DATA STRUCTURE IS RESIZABLE OR GROWABLE ARRAY)

- Exactly same as arrayList
- Insertion order is preserved
- Null insertion is possible
- Duplicate objects can be inserted
- Implements random access so that we can access random objects
- Used mostly for retriving the data
- Not used for insertion and deletion.
- Default capacity = 10
 - Newcapacity= 2*currentcapacity
- Vector **increments 100%** means doubles the array size if the total number of elements exceeds than its capacity

WHAT IS LIST?

1. List is a child interface of collection interface . The implementation classes for List are
 1. Array List , Linked List , Stack and Vector
2. It is an ordered collection of objects
3. it contains duplicate elements
4. It can access the elements randomly
5. Insertion order is preserved
6. Null values will be permitted



MAX ELEMENTS IN THE LIST

```
class Codinghub
{
    public static void main(String[] args)
    {
        int [] arr=new int[] {1,2,3,45,1,2,56,3,4,7,561,2,34,45};
        List<Integer> l=new ArrayList<Integer>();
        for(int i=0;i<arr.length;i++)
            l.add(arr[i]);
        System.out.println("Max element is = "+ Collections.max(l));
    }
}
```


STACK

1. It is the implementation class for list
2. It is a Linear Data Structure & Stores the elements contiguously
3. We will use push() and pop() to add and remove the elements
4. Using peek() we can fetch the top of the stack
5. Applications of Stacks:

We will use stack for method calls(refer runtime stack mechanism in exception handling)

QUEUE

1. It implements the collection interface
2. In queues elements are stored and accessed in First in First Out order
3. Elements are added from the behind and removed from the front



Example :

```
Queue q=new LinkedList();
```

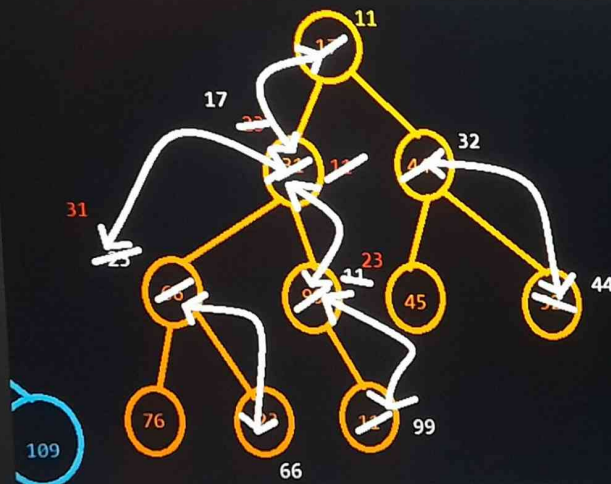
```
q.add(1); and so on....
```

Heap Sort
Min Heap

17, 31, 44, 66, 99, 45, 32, 76, 23, 11

Only a Minimum element must be placed
as a root or parent node

11, 17, 32, 31, 23, 45, 44, 76, 66, 99



PRIORITY QUEUE

1. It implements the queue interface
2. Provides the functionality of the heap data structure
3. Unlike normal queues, priority queue elements are not retrieved in sorted order as per the heap data structure
4. Priority queue does not allow null values



```
6 public static void main(String[] args)
7 {
8     String a[] = new String[] {"Elephant", "Apple", "cat", "Dog", "ball", "fox"};
9     ArrayList<String> al = new ArrayList<>();
10    int x, y;
11    for(x=0; x<a.length; x++)
12    {
13        al.add(a[x]);
14    }
15    al.sort(Comparator.naturalOrder());
16    Iterator it = al.iterator();
17    while(it.hasNext())
18    {
19        System.out.println(it.next());
20    }
21 }
```

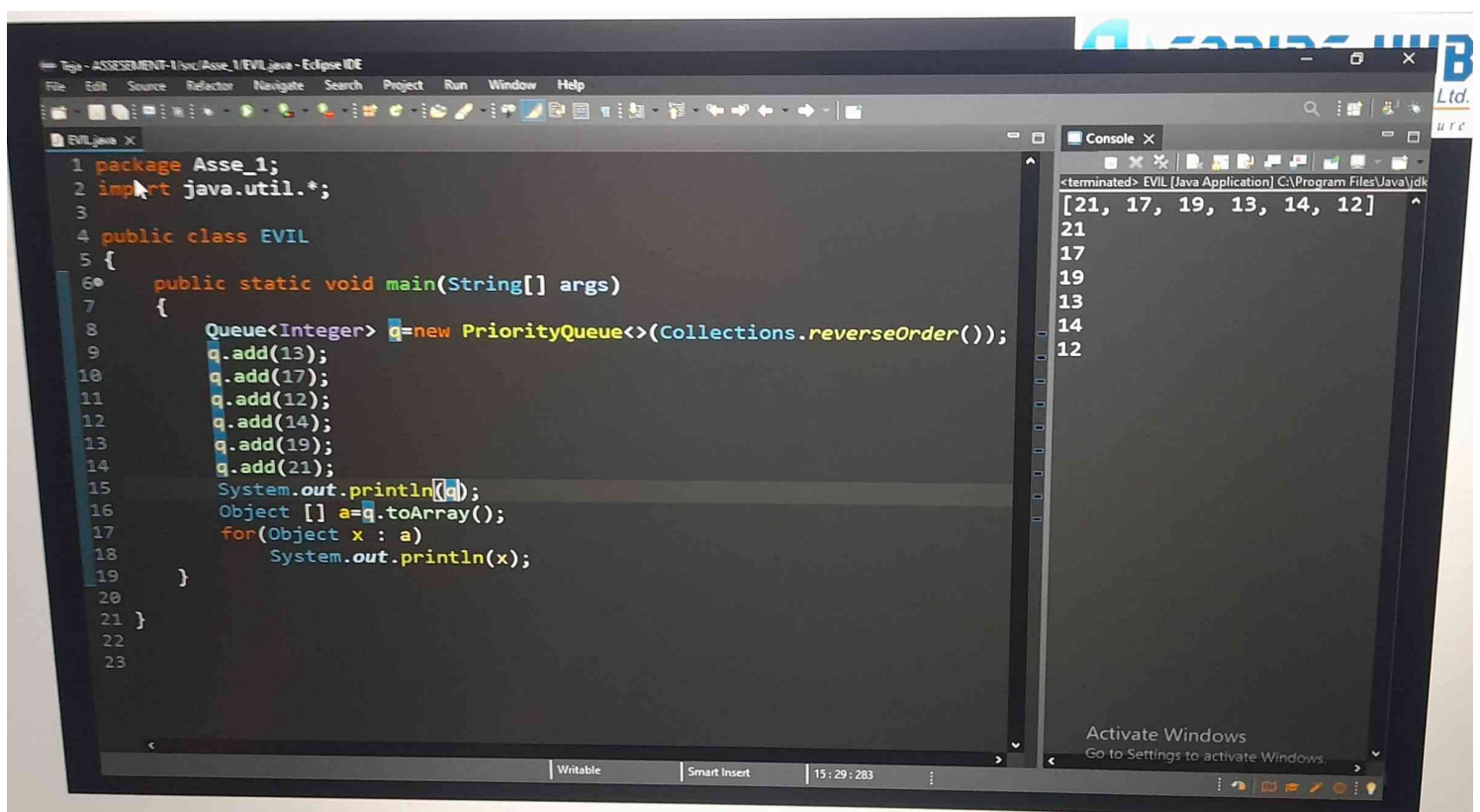
Activate Window

38

Adding elements via ListIterator

```
public class Sample
{
    public static void main(String args[])
    {
        ArrayList<Integer> al=new ArrayList<>(Arrays.asList(10,20));
        ListIterator<Integer> itr=al.listIterator();
        while(itr.hasNext())
        {
            System.out.println(itr.next());
        }
        itr.add(11);
        System.out.println(al);
    }
}
```

```
4 class Demo
5 {
6     public static void main(String args[])
7     {
8         String []a=new String[] {"HELLO","all","good","G","Morning","Coding","Hub"};
9         ArrayList al=new ArrayList();
10        for(int x=0;x<a.length;x++)
11            al.add(a[x]);
12        al.sort(Comparator.naturalOrder());
13        System.out.println(al);
14        al.sort(Comparator.reverseOrder());
15        System.out.println(al);
16    }
17 }
```

DEQUE (STACK + QUEUE)

1. **Java Deque interface is a linear collection that supports element insertion and removal at both the ends.**
2. **Deque is an acronym for “double ended queue”**
3. **It implants the properties of both stack and queue so it can perform LIFO AND FIFO**



Stack

Top
(insertion, deletion)



Queue

Front
(deletion)



Rear
(insertion)

Deque

Front
(insertion, deletion)



Rear
(insertion, deletion)

codinghub.org.in

ARRAY DEQUE

1. Java **ArrayDeque** class provides a resizable array implementation of Deque interface.
2. It has no capacity restrictions. So, it can grow according to the need to handle elements added to the collection.
3. Null elements are restricted in the **ArrayDeque**.
4. **ArrayDeque** class performs faster operations than Stack when used as a stack.
5. **ArrayDeque** class performs faster operations than Linked List when used as a queue.

HASH SET(UNDERLYING DATA STRUCTURE IS HASHTABLE)

1. Hash Set extends Abstract Set and implements the Set interface.
2. It creates a collection that uses a hash table for storage.
3. A hash table stores information by using a mechanism called **hashing**
4. Hash Set contains unique elements only.
5. Hash Set allows null value. Hash set follows the hashing mechanism.
6. Hash Set doesn't maintain the insertion order. Here, elements are inserted on the basis of their hash code.
7. Hash Set is the best approach for search operations.
8. The initial default capacity of Hash Set is 16.

0	32
1	17
2	
3	19 → 99
4	
5	37
6	
7	
8	
9	
10	10
11	11 → 107
12	44 → 76
13	
14	
15	

HashFunction =16

```
int[] a=new int[] {10,17,19,32,44,11,76,37,99,107};
```

Indecies

$10\%16=10$
 $17\%16=1$
 $19\%16=3$
 $32\%16=0$
 $44\%16=12$
 $11\%16=11$
 $76\%16=12$ COLLISION
 $37\%16=5$
 $99\%16=3$ COLLISION
 $107\%16=11$ COLLISION

TOP TO BOTTOM ORDER
32, 17, 19, 99, 37, 10, 11, 107, 44, 76

what is Hashing?

1. HashSet internally uses hash table to store the elements. Elements in hash table are present in key and value pairs
2. Hashing is the process of converting an object into an integer value(hash code) using a hash function($\text{integer} \% 16$)
3. This hash code is used to determine the index(bucket) where the object will be stored in the hash table.
4. when we add an element to the set, its hash function will be calculated and the element is placed in the appropriate index based.
5. Hashing in hashset allows for efficient storage and retrieval of elements especially when dealing with large collections

LOAD FACTOR

1. The load factor is a measure of how full a hash table is. The load factor represents the percentage of the table that is occupied by the entries.
2. $\text{Load factor} = \frac{\text{Number of Entries}}{\text{Table size}}$
3. For example, if a hash map has 10 entries and the table size is 16, the load factor is
 - a. $\text{Load factor} = 10/16 = 0.625$
 - b. A load factor of 0.0 means the table is empty
 - c. A load factor of 1.0 means the table is fully occupied
4. when the load factor exceeds a certain threshold (usually 0.75) the collection is automatically resized the table to maintain performance. This is called **Rehashing**

```
4 class Demo
5 {
6     public static void main(String args[])
7     {
8         HashSet<String> s=new HashSet<String>();
9         s.add("RED");
10        s.add("GREEN");
11        s.add("BLUE");
12        s.add("PINK");
13        s.remove("RED");
14        System.out.println(s);
15    }
16 }
```

Console X

terminated> Demo (2) [Java Application] E:\Recommended\java\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32
PINK, BLUE, GREEN]

LINKED HASH SET (HASH TABLE + LINKED LIST)

Java Linked Hash Set class is a Hash table and Linked list implementation of the Set interface. It inherits the Hash Set class and implements the Set interface.

1. Java Linked Hash Set class contains unique elements only like Hash Set.
2. Java Linked Hash Set class provides all optional set operations and permits null elements.
3. Java Linked Hash Set class is non-synchronized.
4. Java Linked Hash Set class maintains insertion order.

TREE SET

1. Java Tree Set class contains unique elements only like Hash Set.
2. Java Tree Set class access and retrieval times are quite fast.
3. Java Tree Set class doesn't allow null element.
4. Java Tree Set class maintains ascending order.
5. Java Tree Set class contains unique elements only like Hash Set.



PRINT UNIQUE ELEMENTS IN THE ARRAY

```
3 import java.util.HashSet;
4
5 class Codinghub
6 {
7     public static void main(String[] args)
8     {
9         int[] arr= {2,9,3,0,8,3,3,2,9,1};
10        HashSet<Integer>hs=new HashSet<Integer>();
11        for(int i=0;i<arr.length;i++)
12            hs.add(arr[i]);
13        for(int x:hs)
14            System.out.print(x+ " ");
15    }
16 }
```

PRINT UNIQUE ELEMENTS USING

```
3• import java.util.ArrayList;
4 import java.util.Arrays;
5 import java.util.HashSet;
6 import java.util.Set;
7
8 class Codinghub
9 {
10• public static void main(String[] args)
11 {
12     ArrayList<Integer>numbers=new ArrayList<Integer>(Arrays.asList(1,2,3,3,2));
13     Set<Integer>s=new HashSet<Integer>(numbers);
14     System.out.println(s);
15 }
16 }
```

PRINT UNIQUE ELEMENTS USING SET

```
public class Simple
{
    public static void main(String args[])
    {
        int[] a=new int[] {10,20,30,40,10,40,20,67,55};
        List<Integer> al=new LinkedList<>();
        for(int temp : a)
            al.add(temp);
        for(Integer temp1 : al)
        {
            if(al.indexOf(temp1)==al.lastIndexOf(temp1))
            {
                System.out.println(temp1);
            }
        }
    }
}
```


DUPLICATE ELEMENTS IN THE ARRAY

```
public class Simple
{
    public static void main(String args[])
    {
        int[] a=new int[] {10,20,30,40,10,40,20,67,55};
        List<Integer> al=new LinkedList<>();
        Set<Integer> set=new HashSet<>();
        for(int temp : a)
            al.add(temp);
        for(Integer temp1 : al)
        {
            if(al.indexOf(temp1)!=al.lastIndexOf(temp1))
            {
                set.add(temp1);
            }
        }
        System.out.println(set);
    }
}
```

```
2 public class Test
3 {
4     public static void main(String args[])
5     {
6         Integer a[] = {10,20,30,10,20,50,60,60,70};
7         ArrayList<Integer> al=new ArrayList<>(Arrays.asList(a));
8         LinkedHashSet<Integer> s=new LinkedHashSet<>();
9         for(int x=0;x<al.size();x++)
10        {
11            if(al.indexOf(al.get(x))!=al.lastIndexOf(al.get(x)))
12                s.add(al.get(x));
13        }
14        System.out.println("Duplicate Elements are "+s);
15    }
16 }
17 |
```

Console X

<terminated> Test [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (Dec 22, 2023, 5:19:19 PM - 5:19:19 PM) [pid: 11020]
Duplicate Elements are [10, 20, 60]

```

public class Two
{
    public static void main(String args[])
    {
        int[] a=new int[] {10,20,30,40,50,60,70,80,90,100};
        ArrayList<Integer> al=new ArrayList<>();
        for(int x=0;x<a.length;x++)
            al.add(a[x]);
        System.out.println(Collections.max(al));
        System.out.println(Collections.min(al));
        System.out.println("RIGHT ROTATION FOR 2 TIMES");
        Collections.rotate(al, 2);
        System.out.println(al);
        System.out.println("LEFT ROTATION FOR 2 TIMES");
        Collections.rotate(al, -2);
        System.out.println(al);
    }
}

```

HASHSET VS LINKEDHASHSET VS TREESET

```
6 class Codinghub
7 {
8     public static void main(String[] args)
9     {
10         System.out.println("IM HASH SET");
11         Set<String> s=new HashSet<String>(Arrays.asList("hello","all","good","morning","hello"));
12         System.out.println(s);
13         System.out.println("IM LINKED HASHSET");
14         Set<String> s1=new LinkedHashSet<String>(Arrays.asList("hello","all","good","morning","hello"));
15         System.out.println(s1);
16         System.out.println("IM TREESET");
17         Set<String> s2=new TreeSet<String>(Arrays.asList("hello","all","good","morning","hello"));
18         System.out.println(s2);
19     }
20 }
```

Activate Windows
Go to Settings to activate Windows

```
2 import java.util.Set;
3
4 class while_Demo
5 {
6     public static void main(String args[])
7     {
8         int[] a=new int[] {1,2,3,4,3,4,5};
9         int[] b=new int[] { 4,2,6,7};
10        int i,j;
11        Set<Integer> h=new HashSet<Integer>();
12        for(i=0,j=0;i<a.length;i++,j++)
13        {
14            h.add(a[i]);
15            if(j<b.length)
16                h.add(b[j]);
17        }
18        System.out.println(h);
19    }
20 }
```

```
3 class Hello
4 {
5     public static void main(String[] args)
6     {
7         Deque<Integer> q=new ArrayDeque();
8         q.add(10);
9         q.add(20);
10        q.add(30);
11        q.add(40);
12        q.add(35);
13        q.add(22);
14        Iterator i=q.descendingIterator();
15        while(i.hasNext())
16            System.out.println(i.next());
17    }
18 }
```

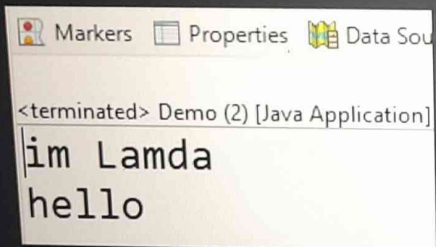
Activate Windows
Go to Settings to activate Windows.

```
5 public class Hello
6 {
7     public static void main(String[] args)
8     {
9         String s="hello all good Morning";
10        String s1[]=s.split(" ");
11        List<String> l=new ArrayList<>();
12        for(int x=0;x<s1.length;x++)
13        {
14            l.add(s1[x]);
15        }
16        l.sort(Comparator.reverseOrder());
17        System.out.println(l);
18    }
19 }
```



```
4 public class Hello
5 {
6     public static void main(String[] args)
7     {
8         int[] a=new int[] {52,2,3,4,1,5,6,23};
9         ArrayList<Integer> l=new ArrayList<>();
10        for(int y=0;y<a.length;y++)
11        {
12            l.add(a[y]);
13        }
14        l.sort(Comparator.naturalOrder());
15        System.out.println(l);
16        l.sort(Comparator.reverseOrder());
17        System.out.println(l);
18    }
19 }
```

FUNCTIONAL INTERFACE



Markers Properties Data Source

<terminated> Demo (2) [Java Application]

```
im Lamda  
hello
```

```
interface X  
{  
    void show();  
}  
interface Y  
{  
    void add();  
}  
class Demo  
{  
    public static void main(String args[])  
    {  
        X x=new X()  
        {  
            public void show()  
            {  
                System.out.println("hello");  
            }  
        };  
        Y y= ()->  
        {  
            System.out.println("im Lamda");  
        };  
        y.add();  
        x.show();  
    }  
}
```

```
10 public static void main(String[] args)
11 {
12     Student1 s1=new Student1(1,'a',"yyy");
13     Student1 s2=new Student1(2,'b',"zzz");
14     ArrayList<Student1> al=new ArrayList<>();
15     al.add(s1);
16     al.add(s2);
17     System.out.println(al);
18 }
19 }
20 class Student1
21 {
22     int roll;char section;String name;//instance
23     Student1(int roll,char section , String name)//local
24     {
25         this.roll=roll;
26         this.section=section;
27         this.name=name;
28     }
29     public String toString()
30     {
31         return name+" "+ section+" "+ roll;
32     }
33 }
```

WHY GO TO COMPARABLE & COMPARATOR?

- We all are familiar with sorting data of primitive data types like int, String, char, etc. We can pass the array to the Arrays.sort() method directly, and Java takes care of sorting those values because Java is aware of how to sort numbers, characters, or strings.
- But what about the custom objects we create like Student, Employee and so on. Java is unaware of how to sort these custom objects, and we need to provide the logic to sort these custom objects explicitly.
- Java provides two interfaces, Comparable and Comparator, that can be used to sort custom objects and objects based on multiple data members

COMPARATOR INTERFACE

- A comparator interface is used to order the objects of user-defined classes. A comparator object is capable of comparing two objects of the same class
- Suppose we have an Array/ArrayList of our own class type, containing fields like roll no, name, address, DOB, etc, and we need to sort the array based on Roll no or name?

COMPARABLE INTERFACE

- A comparable interface is used to order the objects of user-defined classes. A comparable object is capable of comparing single object and current object of the same class
- Suppose we have an Array/ArrayList of our own class type, containing fields like roll no, name, address, DOB, etc, and we need to sort the array based on Roll no or name?

COMPARABLE

```
6 public class App
7 {
8     public static void main( String[] args )
9     {
10         Employee e1=new Employee(11,"raju","cse");
11         Employee e2=new Employee(9,"ramu","it");
12         Employee e3=new Employee(3,"ravi","eee");
13         List<Employee> set=new ArrayList<>();
14         set.add(e1);
15         set.add(e2);
16         set.add(e3);
17         Collections.sort(set);
18         System.out.println(set);
19     }
20 }
```



```

21 class Employee implements Comparable<Employee>
22 {
23     int id;
24     String name;
25     String dept;
26     public Employee()
27     {
28         super();
29     }
30     public Employee(int id, String name, String dept) {
31         super();
32         this.id = id;
33         this.name = name;
34         this.dept = dept;
35     }
36     @Override
37     public String toString()
38     {
39         return "Employee [id=" + id + ", name=" + name + ", dept=" + dept + "]\n";
40     }
41     @Override
42     public int compareTo(Employee o)
43     {
44         return Integer.compare(this.id, o.id); //
45     }
46 }

```

Comparator



```
8 public class B01
9 {
10     public static void main(String[] args)
11     {
12         Studentss s1=new Studentss("aaa",10,'a');
13         Studentss s2=new Studentss("ccc",17,'c');
14         Studentss s3=new Studentss("zzz",9,'b');
15         Studentss s4=new Studentss("bbb",16,'e');
16         ArrayList al=new ArrayList();
17         al.add(s1);
18         al.add(s2);
19         al.add(s3);
20         al.add(s4);
21         Collections.sort(al,new Sortname());
22         System.out.println(al);
23     }
24 }
```

```
25 class Studentss
26 {
27     String name;int roll;char section;
28     public Studentss(String name, int roll, char section) {
29         super();
30         this.name = name;
31         this.roll = roll;
32         this.section = section;
33     }
34     @Override
35     public String toString() {
36         return name + " " + roll + " " + section ;
37     }
38 }
```

```

public class B01
{
    public static void main(String[] args)
    {
        Studentss s1=new Studentss("aaa",10,'a');
        Studentss s2=new Studentss("ccc",17,'c');
        Studentss s3=new Studentss("zzz",9,'b');
        Studentss s4=new Studentss("bbb",16,'e');
        ArrayList al=new ArrayList();
        al.add(s1);
        al.add(s2);
        al.add(s3);
        al.add(s4);
        Collections.sort(al,new Sortname());
        System.out.println(al);
    }
}

```

```

25 class Studentss
26 {
27     String name;int roll;char section;
28     public Studentss(String name, int roll, char section) {
29         super();
30         this.name = name;
31         this.roll = roll;
32         this.section = section;
33     }
34     @Override
35     public String toString() {
36         return name + " " + roll + " " + section ;
37     }
38 }

```

```
39 class Sortroll implements Comparator<Studentss>
40 {
41
42     @Override
43     public int compare(Studentss o1, Studentss o2)
44     {
45
46         return Integer.compare(o1.roll, o2.roll);
47     }
48 }
49
50 class Sortname implements Comparator<Studentss>
51 {
52
53     @Override
54     public int compare(Studentss o1, Studentss o2) {
55         // TODO Auto-generated method stub
56         return o1.name.compareTo(o2.name);
57     }
58 }
59 }
```

```

import java.util.*;
import java.util.function.Consumer;
public class CodingHub
{
    public static void main(String[] args)
    {
        Student s1=new Student(1,"ramu",'a');
        Student s2=new Student(3,"shyam",'d');
        Student s3=new Student(2,"anand",'c');
        Student s4=new Student(4,"srinu",'b');
        List<Student> al=new ArrayList<Student>(Arrays.asList(s1,s2,s3,s4));
        System.out.println("before sorting rollno's");
        Consumer<String> printing=x->System.out.println(x);
        al.forEach(printing);
        Comparator<Student> rollsort=(Student o1,Student o2)->
        {
            return Integer.compare(o1.roll, o2.roll);
        };
        al.sort(rollsort);
        System.out.println("After sorting rollno's");
        al.forEach(printing);
    }
}

class Student
{
    String name;
    int roll;
    char section;
    Student(int roll,String name,char section)
    {
        this.roll=roll;
        this.name=name;
        this.section=section;
    }
    public String toString()
    {
        return "roll:"+this.roll+"\tname:"+this.name+"\tsection:"+this.section;
    }
}

```

```

before sorting rollno's
roll:1    name:ramu      section:a
roll:3    name:shyam     section:d
roll:2    name:anand     section:c
roll:4    name:srinu     section:b

After sorting rollno's
roll:1    name:ramu      section:a
roll:2    name:anand     section:c
roll:3    name:shyam     section:d
roll:4    name:srinu     section:b

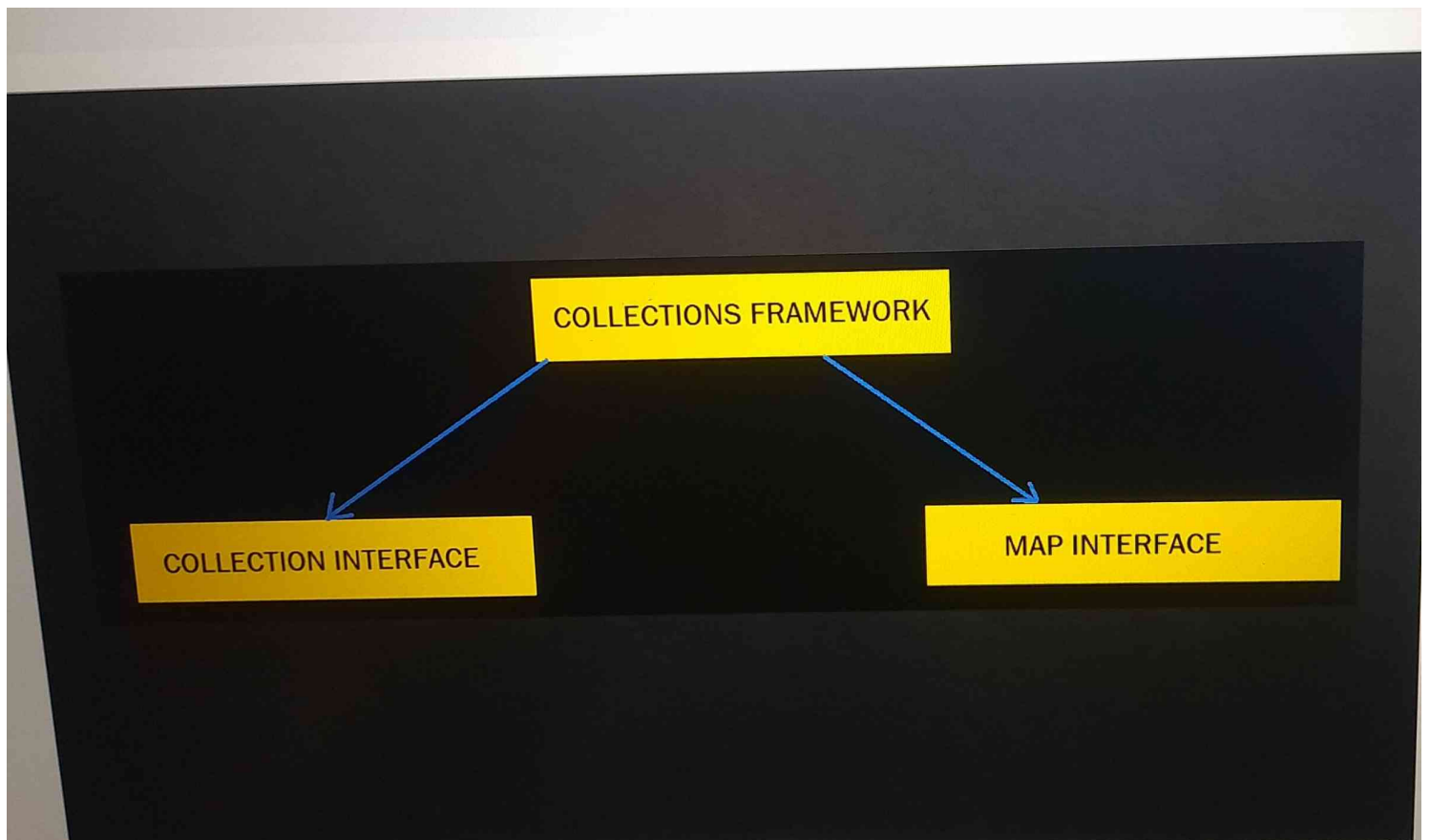
```

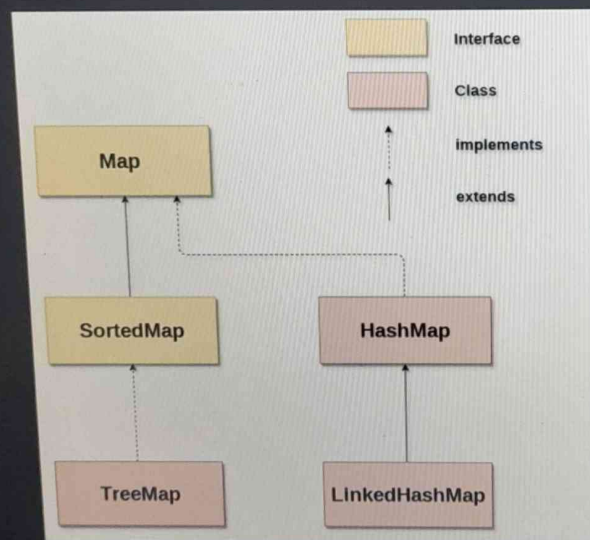
Maps

codinghub.co.in

MAP

1. Maps are part of Collection framework and are used to store data in key-value pairs
2. Each key is associated with exactly one value, and duplicate keys are not allowed, but values can be duplicated
3. Map is mainly used to search, update and delete elements on the basis of keys.
4. Null insertion is may or may not be possible in some classes
5. The Map interface is implemented by classes like
 - a. HashMap
 - b. TreeMap
 - c. LinkedHashMap





Methods in Map Interface

1. `get(object key)` : Retrieves the value for the given key
2. `remove(object key)` : Removes the key-value pair for the given key
3. `containsKey(object key)` : Checks if the map contains the specified key
4. `containsValue(object value)` : Checks if the map contains the specified value
5. `keySet()` : Returns a set of all the keys in the map
6. `values()` : Returns a collections of all values in the map

```
public class Main
{
    public static void main(String[] args)
    {
        Map<Integer,String> map=new HashMap<>();
        map.put(101, "Hyderabad");
        map.put(106, "Vijayawada");
        map.put(99, "Warangal");
        map.put(87, "Delhi");
        System.out.println(map.get(99));
        System.out.println(map.containsKey(187));
        System.out.println(map.containsValue("Delhi"));
        map.remove(87);
        System.out.println(map);
        System.out.println(map.keySet());//all keys
        System.out.println(map.values());//all values
        System.out.println(map.entrySet());//all key-value pairs
    }
}
```

Contd..

- ◆ **HashMap** maps a key and a value. It does not guarantee any order of the elements stored internally in the map.
- ◆ **TreeMap** also maps a key and a value. Furthermore it guarantees the order in which keys or values are iterated - which is the sort order of the keys or values.
- ◆ The **HashMap** implementation is typically the fastest of the two Map implementations, so whenever you don't need to sort the elements in the Map you can just use a **HashMap**. Otherwise use a **TreeMap**.

```

4 class Maps_Example
5 {
6     public static void main(String args[])
7     {
8         Map<Integer,String> m= new HashMap<Integer,String>();
9         m.put(1, "RBI");
10        m.put(2, "HDFC");
11        m.put(3, "SBI");
12        m.put(4, "AndhraBank");
13        //for-each loop
14        for(Map.Entry<Integer,String> temp: m.entrySet())
15        {
16            //print key and values
17            System.out.println(temp.getKey() + " : " + temp.getValue());
18        }
19    }
20 }

```



```

public class Main
{
    public static void main(String args[])
    {
        Map<Integer, String>map=new HashMap<>();
        map.put(11, "Ramu");
        map.put(31, "Raju");
        map.put(87, "Ravi");
        map.put(33, "Ramesh");
        map.put(17, "Raji");
        //Retrieve and print the values
        System.out.println(map.get(31));
        //check if a key exist
        System.out.println(map.containsKey(11));
        //check if a value exist
        System.out.println(map.containsValue("Raju"));
        //Iterating Over entries
        for(Entry e:map.entrySet())
            System.out.println(e.getKey()+"==>"+e.getValue());
        map.remove(12);
        System.out.println(map);
    }
}

```

```

public static void main( String[] args )
{
    HashMap<Integer,String> map3=new HashMap<>();
    map3.put(100, "India");
    map3.put(11, "SriLanka");
    map3.put(31, "Russia");
    map3.put(14, "India");
    int count=0;
    for(Entry<Integer,String> temp:map3.entrySet())
    {
        if(temp.getKey()%2!=0)
            System.out.println(temp.getKey()+" "+temp.getValue());
    }
    for(Entry<Integer,String> temp1:map3.entrySet())
    {
        if(temp1.getValue().equals("India"))
        {
            System.out.println(temp1.getKey());
        }
    }
}

```

getOrDefault()

1. The `getOrDefault()` in java is used to retrieve values that are associated with keys
2. If the key is not present in the map , it returns a default value that you provide, instead of null
 - a. `V getOrDefault(Object key, V defaultValue)`
3. If the key exists in the map , it returns the associated value else it will return the default value

```
public class Main
{
    public static void main(String args[])
    {
        Map<Integer, String> map = new HashMap<>();
        map.put(11, "India");
        map.put(16, "Russia");
        //using getOrDefault()
        String s1 = map.getOrDefault(11, "BHARATH");
        String s2 = map.getOrDefault(12, "Lanka");
        //Srilanka not in the map so returns the default value
        System.out.println("11 ==>" + s1);
        System.out.println("12 ==>" + s2);
    }
}
```

```
public class Main |
{
    public static void main(String[] args)
    {
        String s=new String("hello all good morning hello morning");
        String [] words=s.split(" ");
        Map<String,Integer> map=new HashMap<>();

        for(String temp : words)//hello//all//hello
        {
            map.put(temp,map.getDefault(temp, 0)+1);
        }
        for (Map.Entry<String,Integer> m: map.entrySet())
        {
            System.out.println(m.getKey()+"==>" +m.getValue());
        }
    }
}
```

```

6 class Hello{
7     public static void main(String args[])
8     {
9         // Creating a map using the HashMap
10        Map<String, Integer> numbers = new HashMap<>();
11        // Insert elements to the map
12        numbers.put("One", 1);
13        numbers.put("Two", 2);
14        System.out.println("Map: " + numbers);
15        // Access keys of the map
16        System.out.println("Keys: " + numbers.keySet());
17        // Access values of the map
18        System.out.println("Values: " + numbers.values());
19        // Access entries of the map
20        System.out.println("Entries: " + numbers.entrySet());
21        // Remove Elements from the map
22        int value = numbers.remove("Two");
23        System.out.println("Removed Value: " + value);
24    }

```

hashset Vs hashmap

- ◆ It implements the Set Interface
- ◆ It does not allow duplicate values.
- ◆ While adding an element it requires only one object as a parameter.
- ◆ It is slower than HashMap.
- ◆ It uses the add() method for adding elements.
- ◆ It implements the Map Interface.
- ◆ The key needs to be unique while two different keys can have the same value.
- ◆ While adding an entry, it requires two object values, the **Key** and the **Value** as the parameter.
- ◆ It is faster than HashSet.
- ◆ C

Hashmap vs treemap

- ◆ The Java HashMap implementation of the Map interface is based on hash tables
- ◆ Because HashMap does not order on keys, it allows for heterogeneous elements.
- ◆ A single null key and numerous null values are allowed in HashMap.
- ◆ HashMap does not keep track of any sort of order.
- ◆ Java TreeMap is a Map interface implementation based on a Tree structure.
- ◆ Because of the sorting, TreeMap allows homogenous values to be used as a key.
- ◆ TreeMap does not allow null keys, however multiple null values are allowed.
- ◆ The elements are arranged in chronological sequence (ascending).

HASHSET VS LINKEDHASHSET VS TREESET

```
6 class Codinghub
7 {
8     public static void main(String[] args)
9     {
10         System.out.println("IM HASH SET");
11         Set<String> s=new HashSet<String>(Arrays.asList("hello","all","good","morning","hello"));
12         System.out.println(s);
13         System.out.println("IM LINKED HASHSET");
14         Set<String> s1=new LinkedHashSet<String>(Arrays.asList("hello","all","good","morning","hello"));
15         System.out.println(s1);
16         System.out.println("IM TREESET");
17         Set<String> s2=new TreeSet<String>(Arrays.asList("hello","all","good","morning","hello"));
18         System.out.println(s2);
19     }
20 }
```

codinghub.org.in

Activate Windows

Go to Settings to activate Windows.

17

```

24 src (default package) while_Demo main(String[]): void
4 public static void main(String[] args)
5 {
6     HashMap<Integer, String> h1 = new HashMap<>();
7     h1.put(3, "C");
8     h1.put(1, "A");
9     h1.put(2, "B");
10    System.out.println(h1);
11    LinkedHashMap<Integer, String> h2 = new LinkedHashMap<>();
12    h2.put(3, "C");
13    h2.put(1, "A");
14    h2.put(2, "B");
15    System.out.println(h2);
16    TreeMap<Integer, String> h3 = new TreeMap<>();
17    h3.put(3, "C");
18    h3.put(1, "A");
19    h3.put(2, "B");
20    System.out.println(h3);
21 }
22 }

```

<terminated> while_Demo [Java Ar
 {1=A, 2=B, 3=C}
 {3=C, 1=A, 2=B}
 {1=A, 2=B, 3=C}

```

import java.util.*;
import java.util.Map.Entry;
public class CodingHub
{
    public static void main(String[] args)
    {
        Map<Integer,String> sl=new LinkedHashMap<Integer,String>();
        sl.put(1, "raju");
        // Adds a key-value pair to the map.
        sl.put(2, "gopi");
        sl.put(3, "john");
        sl.put(4, "hari");
        String val1=sl.get(1);//Retrieves the value for the given key.
        String val2=sl.getDefault(7, "not found");
        //Returns the value for the key, or a default value if key is missing.
        boolean val3=sl.containsKey(4);//Checks if the key exists.
        boolean val4=sl.containsValue("hari");//Checks if the value exists.
        String val5=sl.remove(2);//Deletes an entry by key.
        int val6=sl.size();//Returns the number of key-value pairs.
        boolean val7=sl.isEmpty();//Checks if the map is empty
        Set<Integer> val8=sl.keySet();//Returns all keys as a Set.
        Collection val9=sl.values();//Returns all values as a Collection.
        Set<Entry<Integer, String>> entries = sl.entrySet();
        //Returns all key-value pairs as a Set<Map.Entry<K, V>>
        for (Map.Entry<Integer, String> m:entries)
        {
            System.out.println(m.getKey());//Returns all keys from entry.
            System.out.println(m.getValue());//Returns all values from entry.
        }
        sl.forEach((x,y)->System.out.println(x+" "+y));
        String val10=sl.compute(1, (x,y)->y==null?"not found":y.concat(" added"));
        //Updates value dynamically.
        sl.putIfAbsent(5,"ramu");//Adds an entry only if the key is missing.
        sl.replace(3,"daniel");//Updates value for a key if it exists.
        sl.replace(1, "raju", "ravi");//Replaces the value only if it matches oldValue
        sl.clear();//Removes all entries.
    }
}

```

```

4 class Demo
5 {
6     public static void main(String args[])
7     {
8         TreeMap<Integer,String> t=new TreeMap<>();
9         t.put(2, "raju");
10        t.put(12, "ravi");
11        t.put(15, "ramu");
12        t.put(9, "akshay");
13        t.put(5, "arjun");
14        Set<Integer> s=t.keySet();
15        for(Integer temp : s)
16            System.out.println(temp);
17        Collection<String> s1=t.values();
18        for(String temp1 : s1)
19            System.out.println(temp1);
20        Set s3=t.entrySet();
21        for(Object o: s3)
22            System.out.println(o);
23    }
24 }

```

Acti

```

6= public static void main(String args[])
7   {
8       Map<Integer,String> m1=new HashMap<>();
9       m1.put(12, "Rahul");
10      m1.put(34, "vijay");
11      m1.put(23,"jagan");
12      //Retrieve keys
13      Set<Integer>s=m1.keySet();
14      for(Integer temp :s )
15          System.out.println(temp);
16      //Retrieve Values
17      Collection<String> c=m1.values();
18      for(String temp1 : c)
19          System.out.println(temp1);
20  }

```